



Sistema de gestión de mantenimiento para flotas con generación automática de reportes mediante reconocimiento de voz

Trabajo de integración curricular · Documento final

MAQUINARIAS · EQUIPOS · VEHÍCULOS

Autor: Luis Alberto Barragan Villota

Carrera: Desarrollo de Software

Tutor: Yngrid Melo

Instituto Superior Tecnológico Rumiñahui

Septiembre de 2026

Resumen

El presente trabajo desarrolla MEV Mantenimiento, una aplicación Android para registrar y dar seguimiento a actividades ejecutadas sobre maquinarias, equipos y vehículos. El problema abordado es la dispersión de registros operativos y la dificultad de capturar información estructurada en campo. La solución integra autenticación, menú por rol, catálogo de activos, mantenimiento preventivo y correctivo, toma de huella, intervención de llantas, borradores, revisión, historial, registros aprobados, reportes y exportación. Como aporte central se implementó un parser contextual que transforma frases reconocidas por el servicio de voz de Android en comandos verificables para los formularios.

La metodología fue aplicada, tecnológica, iterativa e incremental, adaptando prácticas de Scrum al desarrollo individual. La validación combinó inspección funcional en dispositivo y pruebas unitarias. La compilación de la versión 1.1.0-beta06 se completó correctamente y se ejecutaron nueve pruebas automatizadas sin fallos, enfocadas en el reconocimiento de activos, el dictado compuesto y el rango permitido de huellas. Los resultados demuestran la viabilidad del flujo propuesto y la utilidad de mantener confirmación humana antes de guardar o enviar. No se reportan porcentajes de ahorro ni impacto productivo, porque todavía no se realizó una prueba controlada con operación real.

Palabras clave: mantenimiento de flotas, Android, reconocimiento de voz, trazabilidad, Firebase, Jetpack Compose.

Abstract

This project develops MEV Mantenimiento, an Android application for recording and tracking maintenance activities performed on machinery, equipment, and vehicles. It addresses fragmented operational records and the difficulty of capturing structured information in the field. The solution combines authentication, role-based menus, asset catalogues, preventive and corrective maintenance, tire tread measurements, tire interventions, drafts, review, history, approved records, reports, and exports. Its main contribution is a contextual parser that converts Android speech-recognition results into verifiable form commands.

An applied, technological, iterative, and incremental methodology was used, adapting Scrum practices to an individual academic project. Validation combined functional inspection on a physical device and unit tests. Version 1.1.0-beta06 compiled successfully and nine automated tests passed with no failures. The evidence supports the functional feasibility of the workflow and the value of human confirmation before saving or submitting data. Productivity or savings percentages are not claimed because no controlled real-operation study has yet been performed.

Keywords: fleet maintenance, Android, speech recognition, traceability, Firebase, Jetpack Compose.

Contenido

Edición: En Microsoft Word puede actualizar la tabla automática con clic derecho > Actualizar campo.

Resumen	2
Abstract.....	3
Contenido.....	3
Capítulo 1. Planteamiento del problema.....	6
1.1 Contexto.....	6
1.2 Formulación del problema.....	6
1.3 Objetivo general.....	6
1.4 Objetivos específicos.....	6
1.5 Justificación.....	6
1.6 Alcance y limitaciones.....	7
Capítulo 2. Marco conceptual y tecnológico	8

2.1 Gestión del mantenimiento	8
2.2 Flota heterogénea	8
2.3 Trazabilidad	8
2.4 Reconocimiento y procesamiento de voz	8
2.5 Tecnologías seleccionadas	9
Capítulo 3. Metodología y requisitos	10
3.1 Enfoque metodológico	10
3.2 Iteraciones	10
3.3 Actores	10
3.4 Requisitos funcionales	11
3.5 Requisitos no funcionales	11
Capítulo 4. Diseño de la solución	12
4.1 Arquitectura	12
4.2 Colecciones de datos	12
4.3 Diseño del parser	13
4.4 Diseño visual	13
Capítulo 5. Implementación	14
5.1 Entorno de construcción	14
5.2 Mantenimiento	14
5.3 Vulcanización	15
5.4 Revisión, historial y aprobados	17
5.5 Reportes	18
5.6 Validaciones	19
Capítulo 6. Pruebas, resultados y discusión	20
6.1 Estrategia de prueba	20
6.2 Resultado automatizado	20
6.3 Matriz de pruebas funcionales	20
6.4 Resultados funcionales	21

6.5 Discusión	23
Conclusiones	25
Recomendaciones.....	26
Referencias.....	27
Anexo A. Ficha técnica	28
Anexo B. Lista de comprobación de aceptación	29
Anexo C. Guion resumido de demostración	30
Anexo D. Declaración de alcance de evidencia.....	31

Capítulo 1. Planteamiento del problema

1.1 Contexto

La gestión de una flota mixta requiere conservar información sobre mantenimientos, lecturas de kilometraje u horómetro, condición de llantas, responsables y decisiones de revisión. Cuando estos datos se encuentran en formularios separados o se registran después de la jornada, aumentan las posibilidades de omisión, duplicidad y pérdida de trazabilidad.

El trabajo en campo añade restricciones: el usuario puede llevar guantes, desplazarse entre equipos o disponer de poco tiempo para escribir. Por ello, la captura por voz resulta atractiva, pero no debe sustituir la verificación. Los servicios de reconocimiento pueden alterar palabras, números o signos; la interfaz debe convertir la voz en una propuesta editable y mantener al usuario como responsable de confirmar el dato.

1.2 Formulación del problema

¿Cómo centralizar el registro y seguimiento de mantenimiento de una flota heterogénea, facilitando la captura de datos en campo mediante voz sin perder control, revisión y trazabilidad?

1.3 Objetivo general

Desarrollar una aplicación Android para gestionar registros de mantenimiento y vulcanización de maquinarias, equipos y vehículos, incorporando reconocimiento de voz como mecanismo de captura asistida y reportes para consulta operativa.

1.4 Objetivos específicos

- Modelar activos, usuarios y registros de mantenimiento en una base documental.
- Implementar formularios de mantenimiento, toma de huella e intervención de llanta.
- Diseñar un parser contextual que transforme frases en valores de formulario.
- Establecer borradores, envío, aprobación y devolución para asegurar trazabilidad.
- Consolidar indicadores y exportar información filtrada a PDF y Excel.
- Validar las reglas críticas mediante pruebas automatizadas y pruebas en dispositivo.

1.5 Justificación

La propuesta integra procesos que suelen tratarse por separado. El valor académico reside en combinar interfaz móvil declarativa, persistencia en la nube, reconocimiento de voz, reglas de negocio y control de estados en un producto ejecutable. El valor operativo esperado es disponer de información más

estructurada y consultable; este beneficio se presenta como potencial y deberá medirse posteriormente en un piloto real.

1.6 Alcance y limitaciones

Incluido en la beta	Fuera del cierre actual
Inicio de sesión y menú por rol	Recuperación de contraseña y alta completa de cuentas
Mantenimiento y vulcanización	Adjuntos fotográficos en almacenamiento seguro
Borradores, revisión y aprobados	Reglas de seguridad productivas validadas por rol
Reportes y exportación	Integración con ERP o sistema institucional
Pruebas unitarias y en dispositivo	Estudio controlado de tiempos y productividad
Identidad visual e iconos	Publicación firmada en tienda de aplicaciones

Capítulo 2. Marco conceptual y tecnológico

2.1 Gestión del mantenimiento

El mantenimiento preventivo agrupa actividades planificadas para reducir la probabilidad de fallas; el correctivo responde a una anomalía detectada. En ambos casos, la utilidad del registro depende de asociar el activo, la lectura operativa, el trabajo realizado, las observaciones y los documentos de referencia.

2.2 Flota heterogénea

MEV significa maquinarias, equipos y vehículos. Esta clasificación exige evitar reglas rígidas: algunos activos se controlan por kilometraje, otros por horómetro y otros por ambos. De igual forma, la cantidad de posiciones de llanta varía por activo. El catálogo maestro es, por tanto, una fuente de configuración para los formularios.

2.3 Trazabilidad

La trazabilidad se implementa mediante identificador de documento, UID del usuario, correo, marcas de tiempo y estado. BORRADOR permite completar; ENVIADO pone el registro a disposición del revisor; APROBADO confirma aceptación; DEVUELTO conserva el motivo y habilita corrección.

Ciclo de trazabilidad del registro

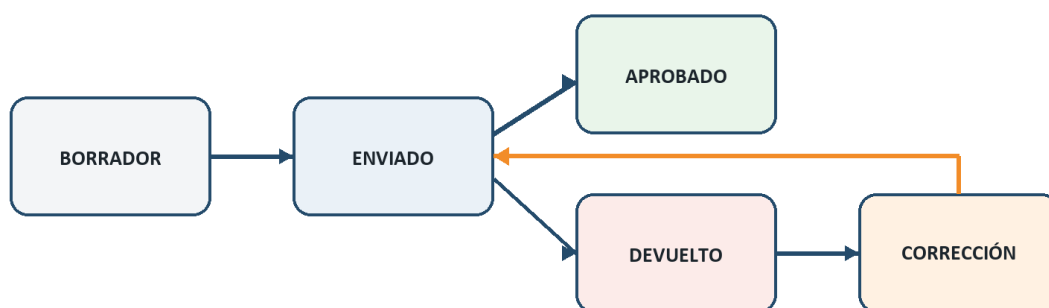


Figura 1. Ciclo de estados utilizado por los registros.

2.4 Reconocimiento y procesamiento de voz

Android SpeechRecognizer proporciona acceso al servicio de reconocimiento del dispositivo y requiere permiso de micrófono. En MEV Mantenimiento el resultado textual pasa por un parser propio. La normalización elimina diferencias de acentuación y espacios, identifica etiquetas, extrae valores,

convierte números expresados con palabras y produce comandos tipados. Esta separación permite probar el parser sin depender del micrófono.

2.5 Tecnologías seleccionadas

Tecnología	Uso en el proyecto	Justificación
Kotlin	Lenguaje principal	Interoperabilidad con Android y sintaxis concisa
Jetpack Compose	Interfaz declarativa	Componentes reutilizables, estado y Material 3
Firebase Authentication	Sesión	Autenticación integrada con SDK Android
Cloud Firestore	Persistencia	Modelo documental y consultas desde la app
SpeechRecognizer	Voz a texto	API disponible en el entorno Android
JUnit	Pruebas	Validación rápida de lógica determinista

Capítulo 3. Metodología y requisitos

3.1 Enfoque metodológico

El proyecto es aplicado y tecnológico: produce un artefacto de software para un problema operativo definido. El proceso fue iterativo e incremental, adaptando Scrum a un desarrollo individual. El Product Backlog se mantuvo mediante historias de usuario; cada incremento incluyó selección, diseño, implementación, validación y corrección.

La definición de terminado exigió que el flujo pudiera ejecutarse, conservara los datos esperados, respetara la navegación por rol y no presentara errores críticos en el escenario probado. Una pantalla visible sin comportamiento conectado no se consideró funcionalidad terminada.

3.2 Iteraciones

Iteración	Objetivo	Resultado al cierre
I1	Problema, alcance y backlog	Requisitos priorizados
I2	Arquitectura y base Android	Proyecto ejecutable
I3	Acceso, perfiles y activos	Sesión y catálogo
I4	Mantenimiento, borradores y voz	Flujo operativo inicial
I5	Huella e intervención	Vulcanización configurable
I6	Historial, reportes y exportación	Consulta consolidada
I7	Correcciones, pruebas, diseño y documentación	Versión beta06

3.3 Actores

Actor	Necesidad principal
Técnico mecánico	Registrar y corregir mantenimientos
Vulcanizador	Registrar huellas e intervenciones
Supervisor	Aprobar o devolver información
Analista	Consultar historial y generar reportes
Planificador	Configurar activos y supervisar el flujo
Jefatura/Gerencia	Consultar información consolidada

3.4 Requisitos funcionales

ID	Requisito	Estado beta06
RF-01	Autenticar usuarios y recuperar su perfil	Implementado
RF-02	Mostrar opciones según rol	Implementado en interfaz
RF-03	Consultar catálogo de activos	Implementado
RF-04	Registrar mantenimiento	Implementado
RF-05	Registrar toma general de huella	Implementado
RF-06	Registrar intervención de llanta	Implementado
RF-07	Guardar y editar borradores	Implementado
RF-08	Aprobar o devolver registros	Implementado
RF-09	Consultar historial y aprobados	Implementado
RF-10	Filtrar y exportar reportes	Implementado
RF-11	Administrar usuarios desde la app	Pendiente
RF-12	Adjuntar evidencia fotográfica	Pendiente

3.5 Requisitos no funcionales

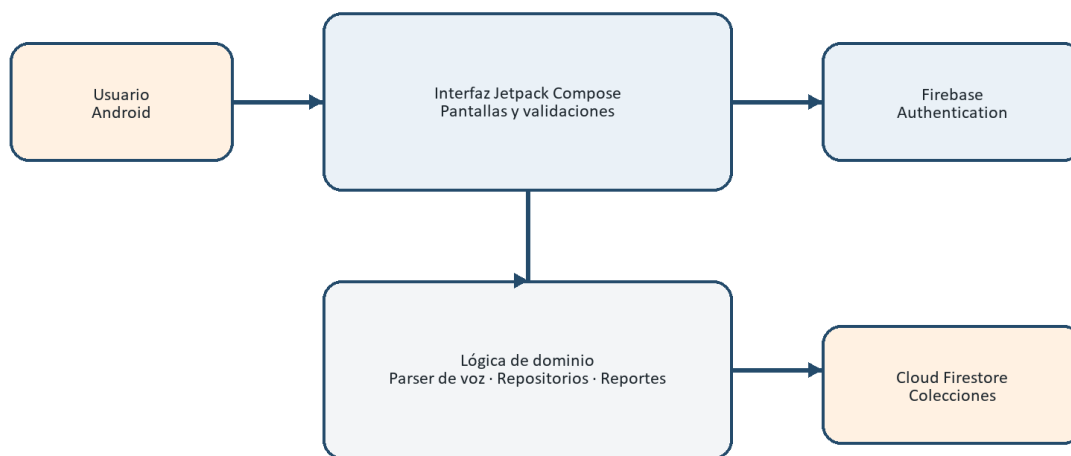
- Usabilidad: formularios legibles, desplazables y con mensajes de confirmación.
- Compatibilidad: Android 8.0 o superior.
- Integridad: validación de campos en interfaz y repositorio.
- Trazabilidad: usuario, estado y marcas temporales en los documentos.
- Mantenibilidad: separación entre pantallas, parser, repositorios y exportación.
- Seguridad: autenticación activa; reglas de Firestore productivas pendientes de endurecimiento.

Capítulo 4. Diseño de la solución

4.1 Arquitectura

La aplicación usa una arquitectura por responsabilidades. Las pantallas Compose gestionan estado y eventos; el parser convierte texto en comandos; los repositorios construyen, validan y persisten documentos; Firebase aporta autenticación y almacenamiento.

Arquitectura funcional de MEV Mantenimiento



La voz se convierte en comandos; el usuario revisa los campos antes de guardar o enviar.

Figura 2. Arquitectura funcional de MEV Mantenimiento.

4.2 Colecciones de datos

Colección	Contenido principal
usuarios	Perfil, cargo, rol, estado y correo
activos	Código, tipo, subtipo, marca, modelo, indicador, lecturas y ubicación
registros_mantenimiento	Servicio, lecturas, trabajo, observaciones, orden, pedido y estado
tomas_huella	Proyecto, lecturas, P1-P12, estado general, novedad, técnico y estado
intervenciones_llanta	Tipo, posición, huella, marca, medida, serie, motivo, técnico y estado

4.3 Diseño del parser

1. Normalizar a minúsculas y retirar acentos.
2. Detectar acciones globales como guardar o enviar.
3. Identificar alias de activo y extraer su secuencia numérica.
4. Localizar etiquetas de campo: proyecto, kilometraje, horómetro, servicio, orden, pedido, huella y técnico.
5. Separar varias etiquetas dentro de una frase.
6. Crear objetos VoiceCommand y aplicarlos en la pantalla activa.
7. Mostrar confirmación y permitir corrección manual.

Frase	Comando
tractor 9	SeleccionarActivo(MTRAC0009)
mini 5	SeleccionarActivo(MMINI0005)
mantenimiento realizado cambio de bandas	ActualizarAccionEjecutada(cambio de bandas)
posición 3 18 milímetros	ActualizarPosicion(3, 18)
guardar borrador	GuardarBorrador

4.4 Diseño visual

La identidad MEV utiliza azul oscuro para confianza y estructura, naranja para energía y trabajo operativo, y fondos de alto contraste. El símbolo integra referencias abstractas a maquinaria, equipos y vehículos sin utilizar logotipos institucionales. Los accesos incorporan iconos por actividad para favorecer reconocimiento visual.

Capítulo 5. Implementación

5.1 Entorno de construcción

Parámetro	Valor
Nombre de paquete	com.luis.mevmantenimiento
Versión	1.1.0-beta06
minSdk / targetSdk	26 / 36
compileSdk	37
Kotlin	2.2.10
Android Gradle Plugin	9.2.1
Firebase BoM	34.15.0
Interfaz	Jetpack Compose + Material 3

5.2 Mantenimiento

El formulario integra activo, tipo de servicio, kilometraje, horómetro, mantenimiento realizado, observaciones, orden de trabajo y número de pedido. La lógica permite guardar como borrador o enviar; los registros devueltos se reabren con su motivo y pueden corregirse.

CNT 9:37

registra una actividad preventiva o correctiva.

Orden de trabajo: OT904.

Voz: orden de trabajo ot-904

Código del activo

VVOLQ0100

VOLQUETA · SHACMAN X3000 - 20M3

Tipo de servicio

Preventivo Correctivo

Kilometraje

56358.0

Horómetro

2358.0

Mantenimiento realizado

Observaciones

banda se arranco

Orden de trabajo

OT904

• Escuchando...

Figura 3. Captura asistida en el módulo de mantenimiento.

5.3 Vulcanización

La toma de huella consulta las posiciones habilitadas para el activo y conserva campos P1 a P12. La intervención registra una acción sobre una posición específica y puede incluir datos de la llanta. Ambos flujos comparten estados y revisión.

CNT 10:09     

Toma general de huella

Comando reconocido:
activo volqueta 60
Activo VVOLQ0060 seleccionado.

VVOLQ0060 - VOLQUETA

Posiciones habilitadas: 10

Proyecto

Kilometraje
254686.0

Horómetro
30076.0

Profundidad de huella por posición

P1 - Huella en mm

P2 - Huella en mm

P3 - Huella en mm

P4 - Huella en mm

• Escuchando...

Figura 4. Formulario de huella con posiciones configuradas por activo.

CNT 10:17

Intervención de llanta

Registra cambios, rotaciones, reparaciones, montajes, desmontajes o bajas.

Intervención enviada correctamente. ID:
3NfLsah57SvAqVIEftvb

Técnico: Jose Camacho.

Voz: nombre del técnico José Camacho

Código del activo
VVLQ0080

VOLQUETA · VOLVO FM- 15M3

Proyecto
Money

Kilometraje
154892.0

Horómetro
10272.0

Tipo de intervención

CAMBIO ROTACIÓN

REPARACIÓN MONTAJE

DESMONTAJE BAJA

Escuchando...

Posición

Figura 5. Formulario de intervención de llanta.

5.4 Revisión, historial y aprobados

La revisión unificada carga documentos ENVIADO de las tres colecciones. Aprobar cambia el estado y registra la fecha; devolver exige un motivo. La consulta de aprobados filtra por APROBADO y el historial del usuario mantiene enviados, aprobados y devueltos.

CNT 10:15

Revisión de registros

Aprueba o devuelve mantenimientos, tomas de huella e intervenciones.

Pendientes: 2

Devolver para corrección

Toma de huella

VVOLQ0099

Proyecto: villomaco

Kilometraje: 67861.0 · Horómetro: 3006.0

Huellas: P2: 15.0 mm · P3: 12.0 mm · P4: 6.0 mm · P5: 8.0 mm · P6: 5.0 mm · P7: 4.0 mm · P8: 2.0 mm · P9: 36.0 mm · P10: 15.0 mm

Estado general: Media vida

Novedad: Corte

Técnico: Abel

Motivo de devolución

Aprobar toma de huella

Devolver para corrección

Volver al menú principal

Figura 6. Pantalla de revisión unificada.

5.5 Reportes

El repositorio de reportes carga mantenimientos, huellas e intervenciones, aplica filtros y calcula indicadores. La exportación PDF produce resumen y detalle; la exportación Excel crea un libro con hojas separadas. Los valores exportados se derivan de los documentos filtrados.

5.6 Validaciones

- Sesión activa antes de escribir en Firestore.
- Activo obligatorio en los flujos enviados.
- Tipo de servicio o intervención cuando corresponde.
- Técnico y estado general obligatorios para huellas enviadas.
- Motivo obligatorio al devolver.
- Huella válida entre 0.1 y 26 mm en interfaz y repositorio.

Capítulo 6. Pruebas, resultados y discusión

6.1 Estrategia de prueba

La validación combinó pruebas unitarias deterministas y pruebas funcionales en un dispositivo Android. Las unitarias se enfocaron en reglas que pueden verificarse sin red: interpretación de voz y rango de huellas. Las funcionales recorrieron formularios, borradores, envío, revisión y consulta.

6.2 Resultado automatizado

Suite	Casos	Fallos	Resultado
ExampleUnitTest	1	0	Aprobado
HuellaValidationTest	4	0	Aprobado
VoiceCommandParserTest	4	0	Aprobado
Total	9	0	BUILD SUCCESSFUL

La ejecución testDebugUnitTest del 1 de septiembre de 2026 finalizó correctamente. Esta evidencia valida los casos codificados, pero no sustituye pruebas de integración de Firebase, rendimiento ni usabilidad.

6.3 Matriz de pruebas funcionales

Caso	Resultado esperado	Evidencia
Tractor 9	Código MTRAC0009	Prueba unitaria aprobada
Mini 5	Código MMINI0005	Prueba unitaria aprobada
Mantenimiento realizado	Completa el campo de trabajo	Prueba unitaria y dispositivo
Dictado compuesto	Separa activo y mantenimiento	Prueba unitaria aprobada
Huella 26	Aceptada	Prueba unitaria aprobada
Huella 27/40	Rechazada	Prueba unitaria y dispositivo
Borrador	Se conserva y puede editarse	Prueba en dispositivo
Aprobar/devolver	Cambia estado; devolución exige motivo	Prueba en dispositivo
Aprobados	Solo muestra APROBADO	Prueba en dispositivo

6.4 Resultados funcionales

- Se centralizaron tres tipos de registro en una aplicación Android.
- El parser reconoce alias de activos de flota mixta y construye códigos con secuencia de cuatro dígitos.
- El dictado puede separar varios campos cuando se expresan con etiquetas reconocibles.
- Los borradores de mantenimiento, huella e intervención se consultan en un centro unificado.
- La revisión distingue pendientes y aprobados, manteniendo el motivo de devolución.
- La validación de huellas se aplica de inmediato y vuelve a comprobarse antes de persistir.
- La identidad visual y los iconos mejoran la diferenciación de módulos.



Figura 7. Evidencia del centro de borradores en dispositivo.

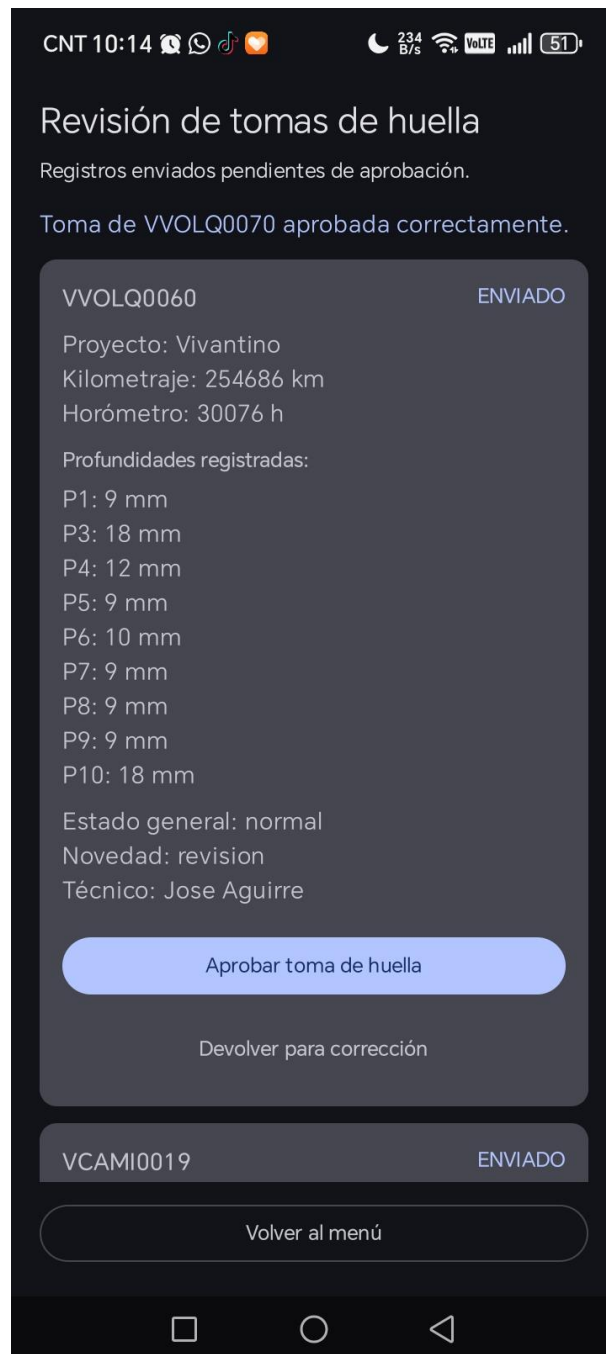


Figura 8. Evidencia del flujo de revisión de huellas.

6.5 Discusión

El resultado respalda la viabilidad del enfoque, pero también confirma que la voz debe tratarse como captura asistida. Las pruebas detectaron errores concretos —alias de activos, mantenimiento realizado y límites de huella— que fueron convertidos en casos automatizados. Este ciclo representa inspección y adaptación, principios consistentes con Scrum.

La arquitectura Firebase acelera el prototipo, aunque traslada una responsabilidad crítica a las reglas de seguridad. La visibilidad del menú por rol mejora la experiencia, pero no constituye autorización suficiente por sí sola. Antes de producción se debe impedir en backend que un usuario ejecute operaciones no permitidas.

No se presentan reducciones porcentuales de tiempo, costos o fallas. Las capturas y pruebas demuestran comportamiento funcional; para afirmar impacto operativo se necesita una línea base, muestra de usuarios, periodo de observación y comparación controlada.

Conclusiones

1. Se desarrolló una beta Android ejecutable que integra mantenimiento, vulcanización, revisión y reportes para una flota heterogénea.
2. La separación entre reconocimiento de voz y parser contextual permitió transformar frases en comandos verificables y probar la lógica sin depender del micrófono.
3. El flujo BORRADOR, ENVIADO, APROBADO y DEVUELTO aporta trazabilidad y habilita correcciones controladas.
4. La configuración por activo resuelve diferencias de indicador y posiciones de llanta entre maquinarias, equipos y vehículos.
5. Las nueve pruebas unitarias aprobadas y las pruebas en dispositivo aportan evidencia del comportamiento central de la versión beta06.
6. La solución cumple el objetivo como prototipo académico; la seguridad productiva, la gestión completa de usuarios y la medición de impacto permanecen como pasos posteriores.

Recomendaciones

1. Diseñar y desplegar reglas de Firestore que autoricen lecturas y escrituras por rol y propietario.
2. Agregar recuperación de contraseña, alta administrada y auditoría de cambios de perfiles.
3. Realizar un piloto con técnicos y vulcanizadores, midiendo tiempo de captura, correcciones y satisfacción.
4. Ampliar la batería con pruebas instrumentadas de Compose, emuladores Firebase y escenarios sin conexión.
5. Incorporar adjuntos fotográficos con validación de formato, tamaño y permisos.
6. Generar una compilación release firmada y establecer política de versiones, respaldo y soporte.

Referencias

Android Developers. (2026). Develop UI with Jetpack Compose.

<https://developer.android.com/develop/ui>

Android Developers. (2026). Meet Android Studio. <https://developer.android.com/studio/intro>

Android Developers. (2026). SpeechRecognizer.

<https://developer.android.com/reference/android/speech/SpeechRecognizer>

Barragan Villota, L. A. (2026a). Propuesta de tesis: Sistema de gestión de mantenimiento para flotas con generación automática de reportes mediante reconocimiento de voz [Documento académico no publicado]. Instituto Superior Tecnológico Rumiñahui.

Barragan Villota, L. A. (2026b). Avance teórico N.º 4: MEV Mantenimiento [Documento académico no publicado]. Instituto Superior Tecnológico Rumiñahui.

Firebase. (2026). Get started with Firebase Authentication on Android.

<https://firebase.google.com/docs/auth/android/start>

Firebase. (2026). Get started with Cloud Firestore. <https://firebase.google.com/docs/firestore/quickstart>

JetBrains. (2025). Kotlin for Android. <https://kotlinlang.org/docs/android-overview.html>

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

Schwaber, K., & Sutherland, J. (2020). La Guía Scrum: La guía definitiva de Scrum.

<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-South-American.pdf>

Anexo A. Ficha técnica

Elemento	Valor
Aplicación	MEV Mantenimiento
Versión	1.1.0-beta06
Paquete	com.luis.mevmantenimiento
APK	MEV-Mantenimiento-1.1.0-beta06-debug.apk
Tamaño APK	25,955,615 bytes
SHA-256	67448607B419202882EA7F13844D944D57AD72339368EB92F868011E3521810E
Rama	LuisB/registros-aprobados-validacion-huellas
Último commit de identidad	161b263 - Integra identidad visual e iconos MEV

Anexo B. Lista de comprobación de aceptación

- ☐ La sesión válida abre un menú acorde al rol.
- ☐ El activo dictado coincide con el código esperado.
- ☐ El mantenimiento puede guardarse como borrador y enviarse.
- ☐ Los registros devueltos muestran el motivo y admiten corrección.
- ☐ Las posiciones de huella coinciden con el activo.
- ☐ Una huella mayor de 26 mm es rechazada.
- ☐ Una intervención exige tipo, posición y técnico al enviar.
- ☐ La devolución exige comentario.
- ☐ La consulta de aprobados excluye enviados y devueltos.
- ☐ Los filtros y exportaciones se ejecutan con datos consistentes.

Anexo C. Guion resumido de demostración

1. Iniciar sesión y explicar el menú por rol.
2. Crear mantenimiento con el dictado: tractor 9 mantenimiento realizado cambio de bandas.
3. Guardar borrador, reabrir, corregir y enviar.
4. Cambiar a rol revisor y aprobar o devolver.
5. Consultar aprobados.
6. Mostrar toma de huella y límite de 26 mm.
7. Aplicar un filtro de reporte y enseñar exportación PDF/Excel.

Plan alternativo: Si falla el servicio de voz, completar manualmente y continuar con un registro previamente enviado. La defensa debe demostrar el flujo, no depender de una única llamada de red.

Anexo D. Declaración de alcance de evidencia

Las figuras de interfaz provienen de pruebas realizadas sobre la aplicación Android durante agosto de 2026. Los datos mostrados son de demostración. Las pruebas unitarias descritas corresponden a la ejecución local del 1 de septiembre de 2026. Este documento no atribuye resultados productivos a esos datos y distingue entre funcionalidad implementada, limitación y trabajo futuro.